

Course ID: SW 302

User Interface (UI) Development

Dr. Mohamed Sami Rakha

Lecture#11

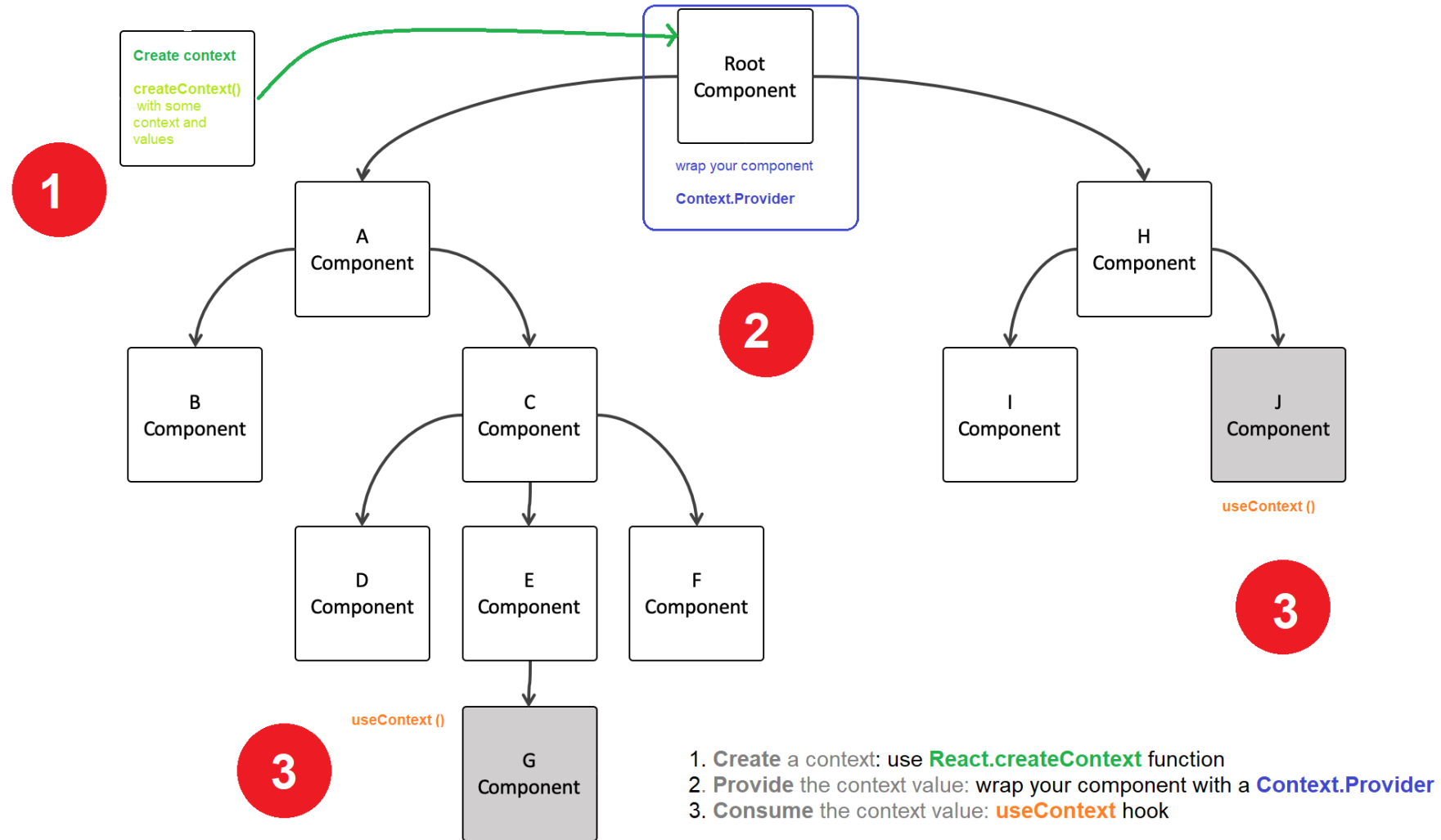
Steps to React Hooks

Lecture Outline

Dr. Mohamed Sami Rakha

React

useContext Hook



1. **Create** a context: use `React.createContext` function
2. **Provide** the context value: wrap your component with a `Context.Provider`
3. **Consume** the context value: `useContext` hook

useContext Hook: Example

In this example, the `MyContext` context is created using the `React.createContext` function, with a default value of `default value`. The `MyComponent` component consumes the context value using the `useContext` hook, and displays the value in the DOM.

App.js

```
1  const MyContext = React.createContext('default value');
2
3  function MyComponent() {
4    const value = useContext(MyContext);
5    return <div>{value}</div>;
6  }
7
8  function App() {
9    return (
10     <MyContext.Provider value="Hello, world!">
11       <MyComponent />
12     </MyContext.Provider>
13   );
14 }
```

Dr. Mohamed Sami Rakha
`useContext(SomeContext)`

```
1  import { useContext } from 'react';
2
3  function MyComponent() {
4    const theme = useContext(ThemeContext);
5    // ..
```

- `useContext` returns the context value for the calling component. It is determined as the value passed to the closest `SomeContext.Provider` above the calling component in the tree.

If there is no such `provider`, then the returned value will be the `defaultValue` you have passed to `createContext` for that `context`. **The returned value is always up-to-date. React automatically re-renders components that read some context if it changes.**

useContext Hook: Example

React automatically re-renders any component that *uses* a context value whenever that context value changes.

When you use:

```
const value = useContext(MyContext);
```

React registers that:

- **This component depends on `MyContext`**
- **If the context's value changes, this component must re-render**

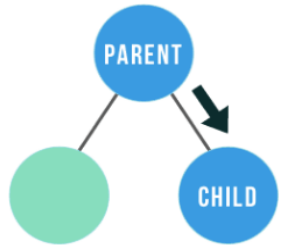
So React updates:

- only the components that *consume* the context [Note!], not the entire tree

Usage: passing data deeply into the tree

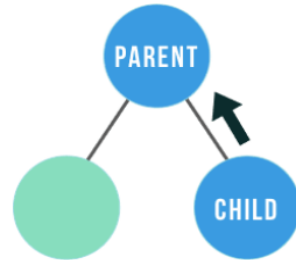
The big issue that comes up is figuring out how components should communicate with each other

Use this as your *table of contents* to jump around and pick the strategy or strategies that apply to your use-case.



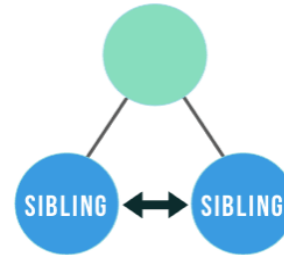
Parent to
Child

1. Props
2. Instance Methods



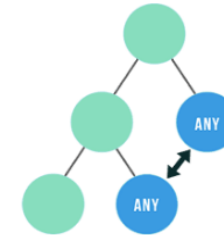
Child to
Parent

3. Callback Functions
4. Event Bubbling



Sibling to
Sibling

5. Parent Component



Any to Any

6. Observer Pattern
7. Global Variables
8. Context

A Solution with context

Context works similarly to props, but instead of providing data to a single child, it can be used to provide data to an entire subtree.

Context can only send data down the tree (parent to subtree). It can be paired with callback functions to pass data back up.

useReducer Hook

General Meaning of “Reducer”

A **reducer** is a function that takes some inputs and combines or “reduces” them into a single output.

Think of it as **taking many things** and producing a smaller, simpler result.

Example in plain *JavaScript*:

```
const numbers = [1, 2, 3, 4];  
const sum = numbers.reduce((accumulator, current) => accumulator + current, 0);  
console.log(sum); // 10
```

•Here:

- **Accumulator**: keeps the running total (**state**)
- **Current**: the current array element (**action**)
- Result: a single value (10)

So `reduce` literally **reduces an array to a single value**.

useReducer Hook in React

The **useReducer** hook is based on the same idea:
It manages **state** by “reducing” actions over time into a **new state**.
You provide a **reducer function** and an initial state:

Dr. Mohamed Sami Rakha

```
const initialState = 0;

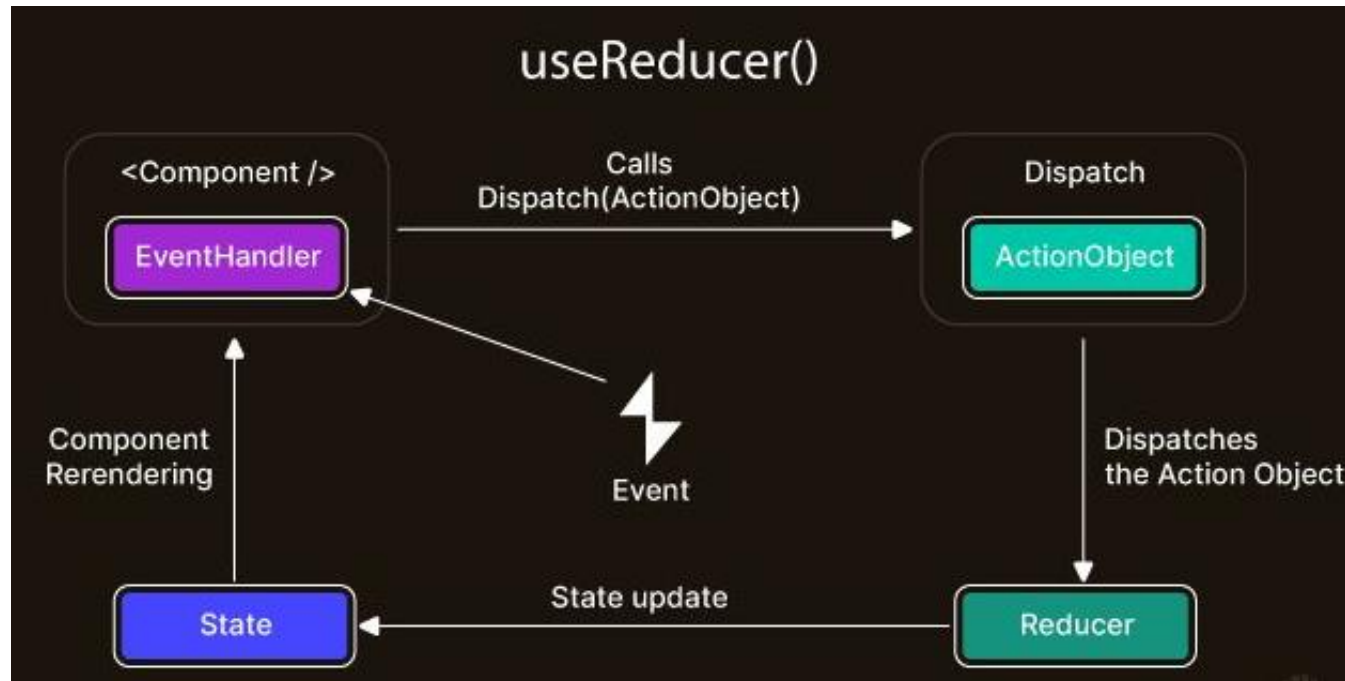
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return state + 1;
    case 'decrement':
      return state - 1;
    default:
      return state;
  }
}

const [count, dispatch] = useReducer(reducer, initialState);
```

- Every time you call `dispatch(action)`, the reducer **takes the current state and the action**, then **returns** a new state.
- Essentially, it “**reduces**” multiple actions over time into the current state.

useReducer Hook

useReducer is a React hook used for managing complex state logic in functional components. It's an alternative to **useState** when state “**transitions**” are more complex or involve “**multiple sub-values**”.



useReducer Hook

useReducer is a Built-in React hook and **part of the standard Hooks API**

Instead of directly setting state, you dispatch actions that describe what should happen. A **reducer** function decides how the state changes based on the **action**.

Syntax: `const [state, dispatch] = useReducer(reducer, initialState);`

reducer: a function (state, action) => return **newState**

initialState: starting state value

state: current state

dispatch: function used to send an action to the reducer

Why Use useReducer:

- Manages **complex state logic** (e.g., multiple state values or dependent updates)
- Makes state changes **predictable**, Easier to test reducer functions separately

When to Use useReducer Instead of useState

- When the state depends on the **previous state**, “**Example:** incrementing or decrementing a counter”.
- When you have **multiple related state values**, “**Example:** a form with several inputs that update together.”
- When you want a **centralized way** to manage updates, “Makes state logic predictable and easier to debug.”
- **When state logic is complex.** “**Example:** conditional updates or multiple actions affecting state.”

Example: Counter

```
import React, { useReducer } from 'react';

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}></button>
      <button onClick={() => dispatch({ type: 'decrement' })}></button>
    </div>
  );
}

export default Counter;
```

useReducer Hook

useState



```
const [loading, setLoading] = useState(false);  
const [error, setError] = useState(false);  
const [post, setPost] = useState({});
```

useReducer



```
const STATE = {  
  loading: false, error: false, post: {},  
};  
const [state, dispatch] = useReducer(reducer, STATE);
```

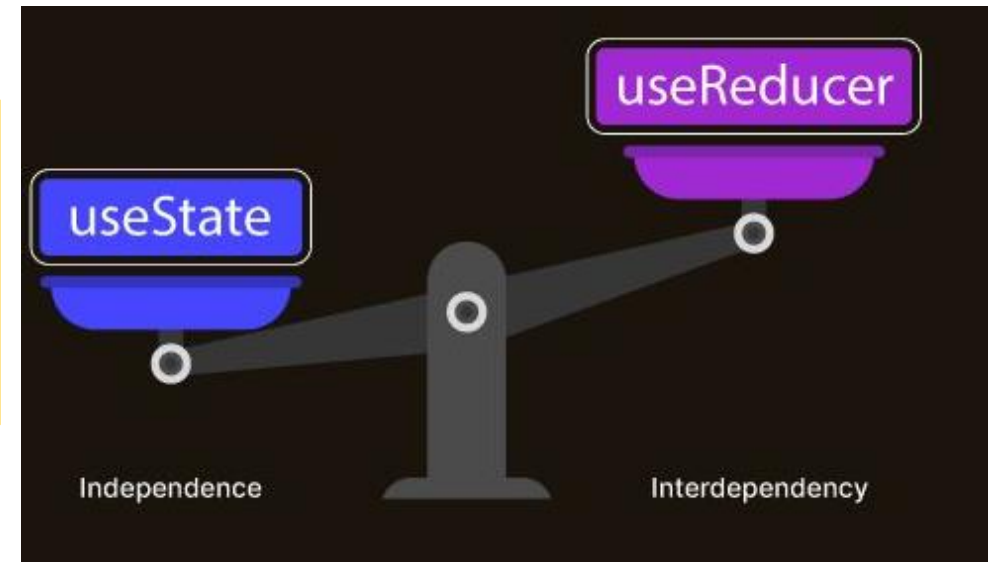
useState vs. useReducer

useState - When Independence Matters

useState is the go-to hook when dealing with independent state elements. It shines when we want to manage multiple state fields that are not interdependent. Its simplicity and ability to handle separate state elements make it an excellent choice for many situations.

useReducer - When State Elements Rely on Each Other

On the other hand, useReducer prevails when state elements rely on one another for updates. When a change in one state affects the transition of another state, useReducer steps up to the plate, ensuring smooth and predictable state management.



useReducer Hook: Todo List Example

This is a Good useReducer Example because: [1] it has complex state (list of objects), [2] includes many action types (add, toggle, delete), and [3] perfectly demonstrates reducer logic

```
import React, { useReducer } from "react";

const initialState = [];

function reducer(state, action) {
  switch (action.type) {
    case "add":
      return [...state, { id: Date.now(), text: action.text, done: false }];

    case "toggle":
      return state.map(todo =>
        todo.id === action.id ? { ...todo, done: !todo.done } : todo
      );

    case "delete":
      return state.filter(todo => todo.id !== action.id);

    default:
      return state;
  }
}
```

"Take all elements inside the state array and copy them into a new array"

...state is the spread operator.

```
export default function TodoApp() {
  const [todos, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <h3>Todo List</h3>

      <button onClick={() => dispatch({ type: "add", text: "New Task" })}>
        Add Todo
      </button>

      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            {todo.text} - {todo.done ? "Done" : "Pending"}

            <button onClick={() => dispatch({ type: "toggle", id: todo.id })}>Toggle</button>
            <button onClick={() => dispatch({ type: "delete", id: todo.id })}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

useContext + useReducer Together

useReducer is great for *state logic*, but it only works **inside one component**.

useContext lets you **share** that reducer-managed state across **the entire app**, without prop drilling.

Together, they create a **mini Redux**:

- global state
- reducer logic
- dispatch actions
- accessible anywhere

This is how **professional React apps** manage global state without using **Redux** [**state management library** used mainly with React to manage **global state**, helps when your app becomes too big for simple **useState**, **useReducer**, or passing props around].

Example

"Global Todo State" using Context + Reducer

Every component in the app can:

- read the todo list
 - add a todo
 - delete a todo
 - toggle a todo
- No props.
 - No passing functions around.
 - Just clean global state.

useContext + useReducer Together

1- Create the Reducer

```
// todoReducer.js
export const initialState = [];

export function todoReducer(state, action) {
  switch (action.type) {
    case "add":
      return [...state, { id: Date.now(), text: action.text, done: false }];

    case "toggle":
      return state.map(todo =>
        todo.id === action.id ? { ...todo, done: !todo.done } : todo
      );

    case "delete":
      return state.filter(todo => todo.id !== action.id);

    default:
      return state;
  }
}
```

2 - Create the Context Provider

This is where we combine `useReducer + useContext`.

```
// TodoContext.js
import { createContext, useReducer } from "react";
import { todoReducer, initialState } from "../todoReducer";

export const TodoContext = createContext();

export function TodoProvider({ children }) {
  const [state, dispatch] = useReducer(todoReducer, initialState);

  return (
    <TodoContext.Provider value={{ state, dispatch }}>
      {children}
    </TodoContext.Provider>
  );
}
```

What this does:

- Initializes global state
- Wraps the whole app
- Provides state and dispatch to every component

useContext + useReducer Together

3- Wrap Your App

```
// App.js
import { TodoProvider } from "../TodoContext";
import TodoList from "../TodoList";
import AddTodo from "../AddTodo";

function App() {
  return (
    <TodoProvider>
      <h2>Global Todo App</h2>
      <AddTodo />
      <TodoList />
    </TodoProvider>
  );
}

export default App;
```

4- Consume Context in Any Component

Add a todo:

```
// AddTodo.js
import { useContext } from "react";
import { TodoContext } from "../TodoContext";

function AddTodo() {
  const { dispatch } = useContext(TodoContext);

  return (
    <button
      onClick={() =>
        dispatch({ type: "add", text: "New Task" })
      }
    >
      Add Todo
    </button>
  );
}

export default AddTodo;
```

5. Display Todos:

```
// TodoList.js
import { useContext } from "react";
import { TodoContext } from "../TodoContext";

function TodoList() {
  const { state, dispatch } = useContext(TodoContext);

  return (
    <ul>
      {state.map(todo => (
        <li key={todo.id}>
          {todo.text} – {todo.done ? "Done" : "Pending"}

          <button onClick={() => dispatch({ type: "toggle", id: todo.id })}>
            Toggle
          </button>

          <button onClick={() => dispatch({ type: "delete", id: todo.id })}>
            Delete
          </button>
        </li>
      ))}
    </ul>
  );
}

export default TodoList;
```



useCallback Hook

What is useCallback?

useCallback is a React Hook that **memorizes a function** so that React does **not recreate it on every render**. React remembers the function instead of creating a new one each time.

Why Do We Need useCallback?

React behavior (important idea)

- Every time a component re-renders -- > **All functions inside it are re-created**
- New function $\neq$ old function (different reference)

This can cause:

- ✗ Unnecessary re-renders
- ✗ Performance problems in large apps

React useCallback Hook Cycle



The **useCallback** hook memoizes a function, ensuring it's recreated only when its dependencies change. It prevents unnecessary re-renders, especially when functions are passed to child components or used in `useEffect`.

Memoizes functions: Avoids re-creation on each render, improving performance.

Preserves function references: Prevents unnecessary re-renders when passed as props to child components.

Reduces computational overhead: **Optimizes performance** in scenarios like filtering or sorting large datasets.

Works with hooks: Ensures stable function references in dependencies, preventing unwanted side effects in `useEffect`.

Use Case

When you: Pass a function as a prop to a child, AND that child is **memoized** (React.memo), AND the function does **not** need to change every render

Dr. Mohamed Sami Rakha

Problem Example (Without useCallback)

```
function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    console.log("Clicked");
  };

  return (
    <>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
      <Child onClick={handleClick} />
    </>
  );
}
```

What happens here?

- Parent re-renders when count changes
- handleClick is recreated every time
- Child receives a **new function reference**
- Child re-renders even if logic didn't change

Solution: with useCallback

```
import { useCallback, useState } from "react";

function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    console.log("Clicked");
  }, []);

  return (
    <>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
      <Child onClick={handleClick} />
    </>
  );
}
```

What useCallback Does?

useCallback(fn, dependencies)

- ✓ Returns a **memoized function**
- ✓ Function is **recreated only when dependencies change**

In this case: `[]`

Means:

Create a function once and never recreate it

useCallback: What Is a “Memoized Child” in React?

A memoized child component is a component that does NOT re-render unless its props change.



Child Component (Memoized)

```
const Child = React.memo(({ onClick }) => {  
  console.log("Child rendered");  
  return <button onClick={onClick}>Child Button</button>;  
});
```

With `useCallback`, the child **does not re-render unnecessarily**

When to Use Memoized Children

- ✓ Large or complex components
- ✓ Lists with many items
- ✓ Components receiving callbacks
- ✓ Performance-critical screens

When SHOULD You Use `useCallback`?

- ✓ Passing callbacks to memoized children
- ✓ Performance-sensitive components
- ✓ Large lists or dashboards
- ✓ `useContext` + reducer apps

useCallback: Several general benefits

useCallback is **not just for memoized child components**. Its main purpose is **to keep a function reference stable across renders**, and that has several general benefits:

Dr. Mohamed Sami Rakha

[1] Stabilizes Function References

Every render normally creates new function objects.

useCallback(fn, deps) ensures the **same function object** is reused unless dependencies change.

[2] Benefits Even Non-Memoized Components

Even if the component using the function **isn't memoized**, a stable function helps:

- Context consumers (like in your TodoProvider)

- Components that rely on reference equality for effects or dependencies (useEffect)

[3] Helps with Context and Props

When functions are passed via **props or context**, using useCallback prevents unnecessary re-renders of **all children consuming those functions**, not just memoized ones.

Makes the app **more predictable and efficient** as it scales.

useCallback is **general-purpose**:

- Improves **performance** by avoiding unnecessary work
- Makes functions **stable** for any consumer
- Prepares your code for **future optimization**

useContext + useReducer + useCallback: Example: Todo App

Why use useCallback here? Because action functions are passed through context and used by many components.

[1] Reducer (State Logic): todoReducer.js

```
export const initialState = [];  
  
export function todoReducer(state, action) {  
  switch (action.type) {  
    case "add":  
      return [...state, action.todo];  
  
    case "toggle":  
      return state.map(todo =>  
        todo.id === action.id  
          ? { ...todo, done: !todo.done }  
          : todo  
      );  
  
    default:  
      return state;  
  }  
}
```

[2] Context + Provider (With useCallback): TodoContext.js

Why useCallback here?

- These functions are:
 - Passed to many components
 - Kept inside context
- Without useCallback:
 - Functions recreated every render
 - All context consumers re-render

```
import { createContext, useReducer, useCallback } from "react";  
import { todoReducer, initialState } from "../todoReducer";  
  
export const TodoContext = createContext();  
  
export function TodoProvider({ children }) {  
  const [state, dispatch] = useReducer(todoReducer, initialState);  
  
  // ✅ Memoized action creators  
  const addTodo = useCallback((text) => {  
    dispatch({  
      type: "add",  
      todo: {  
        id: Date.now(),  
        text,  
        done: false  
      }  
    });  
  }, []);  
  
  const toggleTodo = useCallback((id) => {  
    dispatch({ type: "toggle", id });  
  }, []);  
  
  return (  
    <TodoContext.Provider  
      value={{  
        state,  
        addTodo,  
        toggleTodo  
      }}  
    >  
      {children}  
    </TodoContext.Provider>  
  );  
}
```



useContext + useReducer + useCallback: Example: Todo App

[3] Wrap the App: App.jsx

```
import { TodoProvider } from "../TodoContext";
import TodoInput from "../TodoInput";
import TodoList from "../TodoList";

function App() {
  return (
    <TodoProvider>
      <h1>Todo App</h1>
      <TodoInput />
      <TodoList />
    </TodoProvider>
  );
}

export default App;
```

[4] Add Todo (Consumes Memoized Callback): `addTodo` reference stays stable.

TodoInput.jsx

```
import { useContext, useState } from "react";
import { TodoContext } from "../TodoContext";

function TodoInput() {
  const { addTodo } = useContext(TodoContext);
  const [text, setText] = useState("");

  return (
    <>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <button onClick={() => addTodo(text)}>
        Add
      </button>
    </>
  );
}
```

[5] Memoized Child Component

```
import React from "react";

const TodoItem = React.memo(({ todo, onToggle }) => {
  console.log("TodoItem rendered:", todo.text);

  return (
    <li onClick={() => onToggle(todo.id)}>
      {todo.text} {todo.done ? "✓" : "✗"}
    </li>
  );
});

export default TodoItem;
```

With `useCallback`, the **same function** is passed around → components only re-render when **state actually changes**.

Context stability: Any other components using `addTodo` or `toggleTodo` **don't re-render unnecessarily**.

useContext + useReducer + useCallback: Example: Todo App

[6] Todo List

```
import { useContext } from "react";
import { TodoContext } from "../TodoContext";
import TodoItem from "../TodoItem";

function TodoList() {
  const { state, toggleTodo } = useContext(TodoContext);

  return (
    <ul>
      {state.map(todo => (
        <TodoItem
          key={todo.id}
          todo={todo}
          onToggle={toggleTodo}
        />
      ))}
    </ul>
  );
}

export default TodoList;
```

In context + reducer apps, useCallback prevents action functions from being recreated, so memoized child components don't re-render unnecessarily.

Think of useCallback as **locking the function in memory**:
Without it : every render creates a **new copy** of addTodo →
“ripples” through all context consumers.
With it: the **same function** is passed around → components only re-render when **state actually changes**.

toggleTodo is memoized → child does not re-render unnecessarily.

useMemo Hook

What is useMemo?

`useMemo` is a React Hook that **memoizes a computed value** so it is **not recalculated on every render**.

Difference Between `useCallback` and `useMemo`

Hook

`useCallback`

`useMemo`

Memoizes

a **function**

a **value**

```
useCallback(() => fn, deps)
```

```
useMemo(() => value, deps)
```

Why Do We Need `useMemo`?

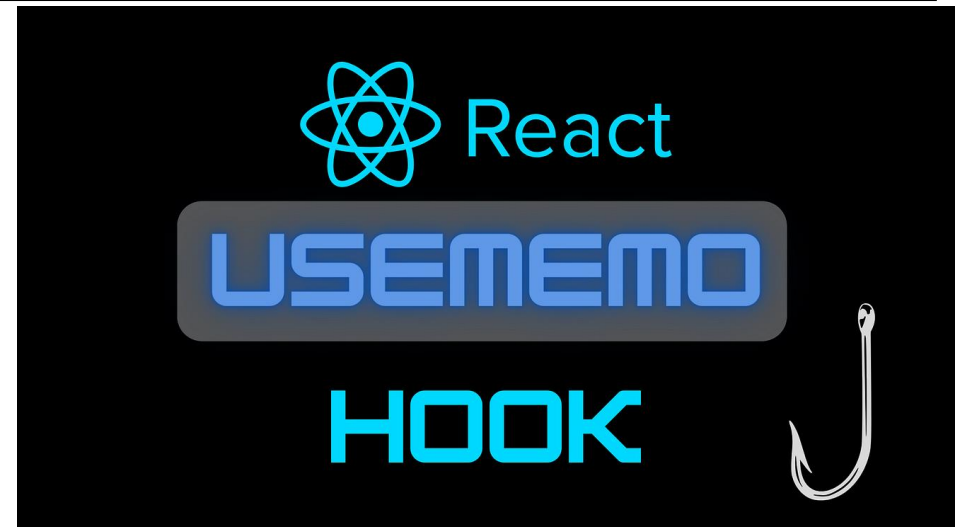
React re-runs:

- All component code
- All calculations inside it
- On **every render**

If a calculation is:

- Expensive
- Repeated
- Unrelated to some state changes

Then, **Performance suffers**.



useMemo: Example

Without useMemo

```
function App() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  const expensiveCalculation = () => {
    console.log("Calculating...");
    let total = 0;
    for (let i = 0; i < 1_000_000; i++) {
      total += i;
    }
    return total;
  };

  return (
    <>
      <input
        value={text}
        onChange={e => setText(e.target.value)}
      />
      <p>{expensiveCalculation()}</p>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </>
  );
}
```

Problem:

- Typing in input triggers re-render
- Expensive calculation runs again

Solution Using useMemo

```
import { useMemo, useState } from "react";

function App() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  const result = useMemo(() => {
    console.log("Calculating...");
    let total = 0;
    for (let i = 0; i < 1_000_000; i++) {
      total += i;
    }
    return total;
  }, []);

  return (
    <>
      <input
        value={text}
        onChange={e => setText(e.target.value)}
      />
      <p>{result}</p>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </>
  );
}
```

Expensive calculation runs **only once**.

Realistic Example (Lists & Filtering)

Without useMemo

```
const filteredTodos = todos.filter(todo => todo.done);
```

With useMemo

```
const filteredTodos = useMemo(() => {  
  return todos.filter(todo => todo.done);  
}, [todos]);
```

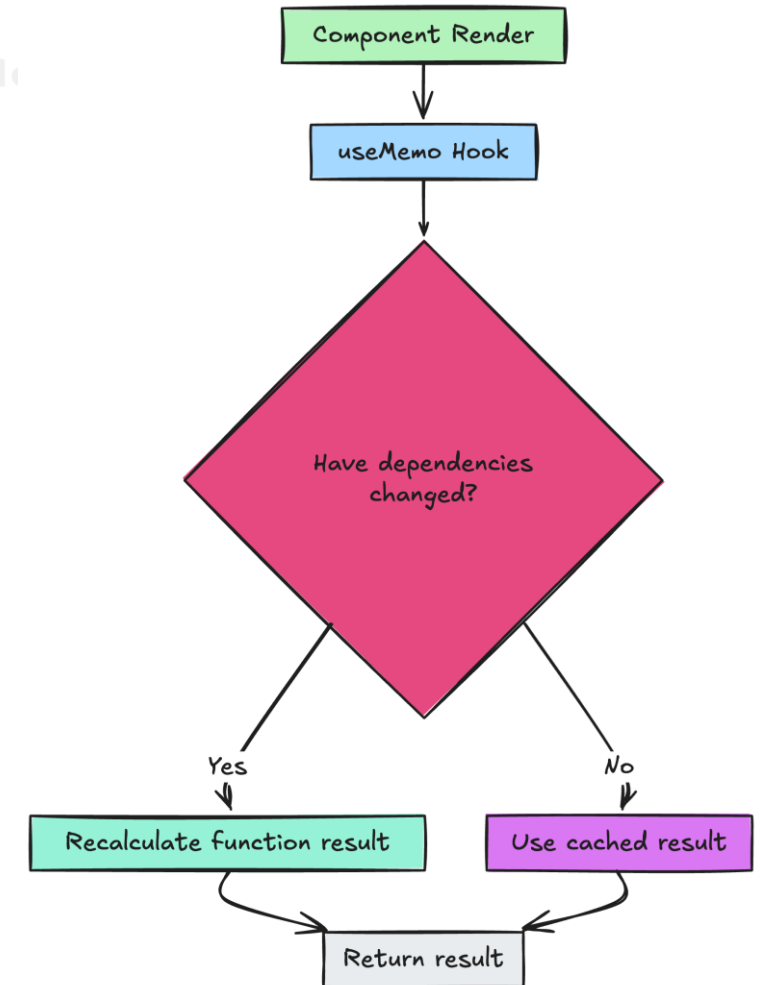
Filters recomputed **only when todos changes**.

When SHOULD Students Use useMemo?

- ✓ Expensive calculations
- ✓ Filtering / sorting large lists
- ✓ Derived state
- ✓ Prevent recalculation on unrelated renders

When NOT to Use It

- Simple calculations
- Small apps



Hooks: Easy Mental Model

Hook	Remembers
useState	Data
useEffect	Side effects
useContext	Shared state
useReducer	Update logic
useCallback	Functions
useMemo	Values



Rakha



Dr. Mohamed Sami Rakha

Thank You...